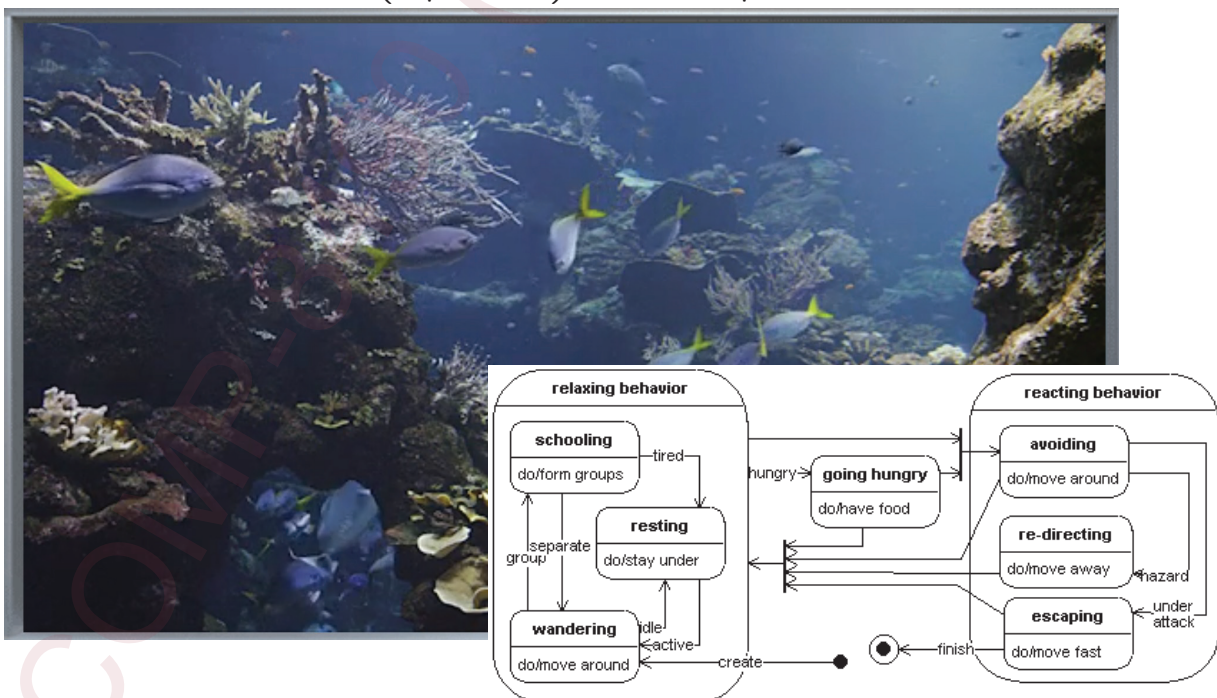


PART I. BEHAVIOR CATEGORIES

1. Behavior categorized by stimulus and object of change

stimulus (reason for change)	object of change			
	TransformGroup (visual objects change orientation or location)	Geometry (visual objects change shape or color)	Scene Graph (adding, removing, or switching objects)	View (change viewing location or direction)
user	interaction	application specific	application specific	navigation
collisions	visual objects change orientation or location	visual objects change appearance in collision	visual objects disappear in collision	View changes with collision
time	animation	animation	animation	animation
View location	billboard	level of detail (LOD)	application specific	application specific

• Virtual fish (aquarium): an example



2. Java 3D behavior support:



- A *Behavior* is a Java class that makes changes to a scene graph
 - In a broad sense, a Java application is a behavior.
 - Java 3D provides a *Behavior* class as a base class for smaller components that change the scene
 - An application extends *Behavior* further to create one or more behaviors to change scene content
 - Often one behavior for each shape being animated

- The *Behavior* class supports arbitrary content changes via Java methods
 - Schedules behaviors to run only when necessary
 - Enables independent behaviors run in parallel
 - Executes animation independent of host speed
- The *Behavior* node contains an enable flag, a scheduling region, a scheduling interval, and a wakeup condition.
 - The *scheduling region* defines a spatial volume that serves to enable the scheduling of *Behavior* nodes.
 - A *Behavior* node is active (can receive stimuli) whenever an active *ViewPlatform*'s activation volume intersects a *Behavior* object's scheduling region.
 - Only active behaviors can receive stimuli.

- The `scheduling interval` defines a partial order of execution for behaviors that wake up in response to the same wakeup condition, i.e., those behaviors that are processed at the same “time”.
 - Given a set of behaviors whose wakeup conditions are satisfied at the same time, the behavior scheduler will execute all behaviors in a lower scheduling interval before executing any behavior in a higher scheduling interval.
 - Within a scheduling interval, behaviors can be executed in any order, or in parallel.
- Wakeup conditions control when to wakeup next
 - Specified on each wakeup

- `WakeupCondition` is an abstract class specialized to different `WakeupCriterion` objects and to combining objects containing multiple `WakeupCriterion` objects.
- Scheduling bounds control scheduling
 - The behavior scheduler invokes the `processStimulus` method if the behavior’s scheduling region intersects an active `ViewPlatform`’s activation volume and all of that behavior’s wakeup criteria are satisfied.



- An example: `TickTockCollision.java`

```

16 public class CollisionDetector extends Behavior {
17     private boolean inCollision = false; private Shape3D shape;
18     private ColoringAttributes shapeColoring;
19     private Appearance shapeAppearance;
20     private WakeupOnCollisionEntry wEnter;
21     private WakeupOnCollisionExit wExit;
22
23     public CollisionDetector(Shape3D s) {
24         shape = s;          // save the original color of 'shape'
25         shapeAppearance = shape.getAppearance();
26         shapeColoring = shapeAppearance.getColoringAttributes();
27         inCollision = false;
28     }
29     public void initialize() {
30         wEnter = new WakeupOnCollisionEntry(shape);
31         wExit = new WakeupOnCollisionExit(shape);
32         wakeupOn(wEnter);          // initialize the behavior
33     }
34     public void processStimulus(Iterator<WakeupCriterion> criteria) {
35         Color3f highlightClr = CommonsXY.Green;
36         ColoringAttributes highlight = new ColoringAttributes(highlightClr,
37             ColoringAttributes.SHADE_GOURAUD);
38         inCollision = !inCollision; // collision has taken place
39
40         if (inCollision) {          // change color to highlight 'shape'
41             shapeAppearance.setColoringAttributes(highlight);
42             wakeupOn(wExit);          // keep the color until no collision
43         } else {                    // change color back to its original
44             shapeAppearance.setColoringAttributes(shapeColoring);
45             wakeupOn(wEnter);          // wait for collision happens
46     }

```

3. Behavior is an abstract class that defines two methods that must be overridden by a subclass:

- `initialize()` allows a Behavior object to initialize its internal state and specify its initial wakeup condition(s).
 - A Behavior object is initialized once when the behavior's containing BranchGroup node is added to the virtual universe.
 - A wakeup condition must be set or else the behavior's `processStimulus` method is never executed.
 - Java 3D does not invoke the `initialize` method in a new thread. Thus, for Java 3D to regain control, the `initialize` method must not execute an infinite loop; it must return.

- The `processStimulus` method performs its computations and actions (possibly including the registration of state change information that could cause Java 3D to wake other Behavior objects), establishes its next wakeup condition, and finally exits.
 - A typical behavior will modify one or more nodes or node components in the scene graph. These modifications can happen in parallel with rendering.
 - All modifications to scene graph objects (excluding geometry by-reference or texture by-reference) made from the `processStimulus` method of a single behavior instance are guaranteed to take effect in the same rendering frame.

- All modifications to scene graph objects (excluding geometry by-reference or texture by-reference) made from the `processStimulus` methods of the set of behaviors that wake up in response to a wakeup condition by `WakeupOnElapsedFrames(0)` are guaranteed to take effect in the same rendering frame,
 - Other than the above two cases, applications cannot count on behavior execution being synchronized with rendering. In particular, modifications to geometry by-reference or texture by-reference are not guaranteed to show up in the same frame as other scene graph changes.

4. Creating scheduling bounds of behavior

- A behavior only needs to be scheduled if viewer is nearby
 - Its scheduling bounds intersects the viewer's volume
 - A behavior's scheduling bounds is a bounded volume
 - Sphere, box, polytope, or combination
 - Use a huge bounding sphere for a global behavior
 - By default, behaviors have no scheduling bounds and are never executed!
 - **Common error:** forgetting to set scheduling bounds
- Anchoring scheduling bounds
 - A behavior's bounding volume can be relative to the behavior's coordinate system:
 - Volume centered on origin

- As origin moves, so does volume
- It can also be relative to a leaf's coordinate system
 - The leaf node can be elsewhere in scene graph
 - As that leaf node moves, so does volume
 - If behavior's origin moves, volume does not

```
private static TransformGroup createColumn(double scale, Vector3d pos) {
    Transform3D transM = new Transform3D();
    transM.set(scale, pos); // Create base TG with 'scale' and 'position'
    TransformGroup baseTG = new TransformGroup(transM);
    Shape3D shape = new Box(0.5, 5.0, 1.0);
    baseTG.addChild(shape); // Create a column as a box and add to 'baseTG'
    Appearance app = shape.getAppearance();
    ColoringAttributes ca = new ColoringAttributes();
    ca.setColor(0.6f, 0.3f, 0.0f); // set column's color and make changeable
    app.setCapability(Appearance.ALLOW_COLORING_ATTRIBUTES_WRITE);
    app.setColoringAttributes(ca);
    CollisionDetector cd = new CollisionDetector(shape);
    cd.setSchedulingBounds(CommonsXY.boundH); // detect column's collision
    baseTG.addChild(cd); // add column with behavior of CollisionDetector
    return baseTG; }
```


5. Waking up a behavior

- `WakeupCriterion` extends `WakeupCondition` to provide multiple ways to wakeup a behavior

Class Hierarchy

```

java.lang.Object
├── org.jogamp.java3d.WakeupCondition
│   └── org.jogamp.java3d.WakeupCriterion
│       ├── org.jogamp.java3d.WakeupOnActivation
│       ├── org.jogamp.java3d.WakeupOnAWTEvent
│       ├── org.jogamp.java3d.WakeupOnBehaviorPost
│       ├── org.jogamp.java3d.WakeupOnCollisionEntry
│       ├── org.jogamp.java3d.WakeupOnCollisionExit
│       ├── org.jogamp.java3d.WakeupOnDeactivation
│       ├── org.jogamp.java3d.WakeupOnElapsedFrames
│       ├── org.jogamp.java3d.WakeupOnElapsedTime
│       ├── org.jogamp.java3d.WakeupOnSensorEntry
│       ├── org.jogamp.java3d.WakeupOnSensorExit
│       ├── org.jogamp.java3d.WakeupOnTransformChange
│       ├── org.jogamp.java3d.WakeupOnViewPlatformEntry
│       └── org.jogamp.java3d.WakeupOnViewPlatformExit

```

- Even when scheduled, a behavior runs only when *wakeup criteria* are met
 - A specified number of frames/time interval has elapsed
 - A specified AWT event occurs
 - The center of a `ViewPlatform` or a specified `Sensor` enters/exits a specified region
 - A behavior is activated/deactivated
 - A specified `TransformGroup` node's transform changes
 - A specified `Shape3D` node's `Geometry` object collides or no longer collides with any other object
 - Movement occurs between a specified `Shape3D` node's `Geometry` object collides with any other object
 - A specified `Behavior` object posts a specific event

- Multiple criteria can be AND/ORed to form *wakeup conditions*
 - `WakeupCondition` extends `Object` and provides several subclasses to group wakeup criterion

Class Hierarchy

```

java.lang.Object
├─ org.jogamp.java3d.WakeupCondition
│   ├─ org.jogamp.java3d.WakeupAnd
│   ├─ org.jogamp.java3d.WakeupAndOfOrs
│   ├─ org.jogamp.java3d.WakeupOr
│   └─ org.jogamp.java3d.WakeupOrOfAnds

```

6. Collision detection in Java 3D

- Java 3D includes a mechanism for collision detection, but it is the programmer's responsibility to implement collision avoidance.

- Collision detection tells when a collision takes place.
- Collision avoidance takes corresponding actions when a collision takes place.
- Collisions are considered a form of behaviour, and two sets of wakeup criteria involve collision check.
 - The center of `ViewPlatform` enters/exits a region.
 - A specified `Shape3D` node's `Geometry` object collides or no longer collides with any other object.
- In both cases, the active object/viewpoint has an activation region surrounding it, and each of the objects marked as collidable have a region surrounding them. If these two areas intersect, the objects are added to the list of active areas to watch for collision.

- The use of bounding (USE_BOUNDS) and a hierarchical tree speeds up detection, and the use of geometric data (USE_GEOMETRY) improves accuracy.

```
public void initialize() {           // USE_GEOMETRY USE_BOUNDS
    wEnter = new WakeupOnCollisionEntry(shape,
        WakeupOnCollisionEntry.USE_GEOMETRY);
    wExit = new WakeupOnCollisionExit(shape,
        WakeupOnCollisionExit.USE_GEOMETRY);
    wakeupOn(wEnter);                // initialize the behavior
}
```

PART II. ANIMATION

1. Computer animation brings life to the virtual world.

- Animation covers all changes that have a visual effect.
 - *motion dynamics*: changes in positions.
 - *update dynamics*: changes in shape, color, transparency, structure, and texture.
 - *changes in lighting, camera position, orientation, focus, and maybe rendering technique*.
- Controlling the attributes results in plenty of techniques:
 - Develop algorithms to generate *curves* and special techniques for forming one seamless curve from a set of jointed short curve segments.

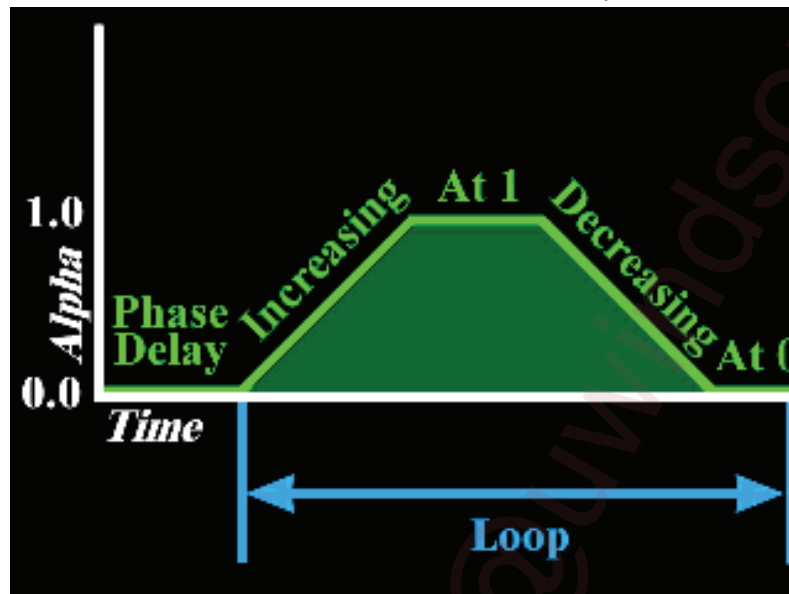
- Study issues of *color* and *realism*, and tools for assigning objects with a wide range of *physical attributes* in the form of color and texture.
- Spend considerable *time* on illuminating the imaginary world, e.g., to decide where each light source is to be positioned, its intensity of color.

2. Java 3D provides *interpolator* for simple behaviors

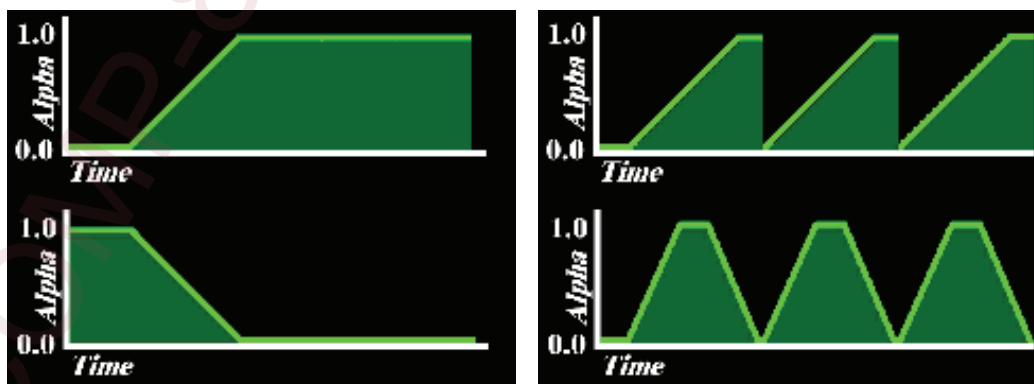
- It allows the change of a parameter from a starting to an ending value during a time interval
- An interpolator uses two mappings:
 - Time-to-*Alpha*
 - *Alpha* is a generalized value that varies from 0.0 to 1.0 over a time interval

- Alpha-to-*Value*
 - Different interpolator types map to different values, such as transforms, colors, switches
- Mapping time to alpha
 - Class `Alpha` extends `Object`
 - `Alpha` methods construct and control alpha start and looping, or get the current value
 - `Alpha` methods also set stage parameters
 - An *Alpha generator* computes alpha using:
 - Trigger time
 - *Phase Delay* before initial alpha change
 - *Increasing time* for increasing alpha
 - *At-one* time for constant high alpha

- Decreasing time for decreasing alpha
- At-Zero time for constant low alpha



- Increasing and decreasing phases may be individually enabled or disabled and their acceleration controlled
 - Increasing ramp controls increasing acceleration
 - Decreasing ramp controls decreasing acceleration
- This model of alpha generalizes to several different types of one-shot and cyclic behaviors



3. Types of interpolators

```

java.lang.Object
├── org.jogamp.java3d.SceneGraphObject
│   ├── org.jogamp.java3d.Node
│   │   ├── org.jogamp.java3d.Leaf
│   │   │   ├── org.jogamp.java3d.Behavior
│   │   │   │   ├── org.jogamp.java3d.Interpolator
│   │   │   │   │   ├── org.jogamp.java3d.ColorInterpolator
│   │   │   │   │   ├── org.jogamp.java3d.PathInterpolator
│   │   │   │   │   │   ├── org.jogamp.java3d.PositionPathInterpolator
│   │   │   │   │   │   ├── org.jogamp.java3d.RotationPathInterpolator
│   │   │   │   │   │   ├── org.jogamp.java3d.RotPosPathInterpolator
│   │   │   │   │   │   └── org.jogamp.java3d.RotPosScalePathInterpolator
│   │   │   │   │   ├── org.jogamp.java3d.PositionInterpolator
│   │   │   │   │   ├── org.jogamp.java3d.RotationInterpolator
│   │   │   │   │   ├── org.jogamp.java3d.ScaleInterpolator
│   │   │   │   │   ├── org.jogamp.java3d.SwitchValueInterpolator
│   │   │   │   │   └── org.jogamp.java3d.TransparencyInterpolator

```

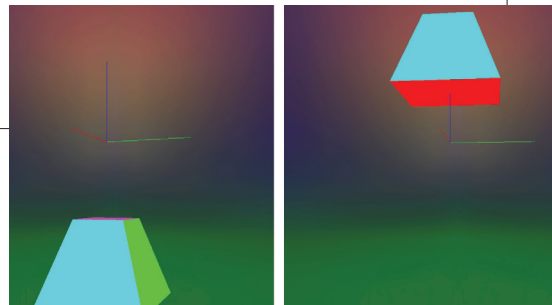
- Simple interpolators map alpha to a value between start and end values
 - Single transforms
 - PositionInterpolator, RotationInterpolator, and ScaleInterpolator
 - Colors and transparency
 - ColorInterpolator and TransparencyInterpolator
 - Switch group values
 - SwitchValueInterpolator
- *Path* interpolators map alpha to a value along a path
 - Single transforms
 - PositionPathInterpolator
 - RotationPathInterpolator

- Combined transforms
 - RotPosPathInterpolator
 - RotPosScalePathInterpolator
- All interpolators specify a *target* into which to write new values, for example,
 - Transform interpolators use a TransformGroup target
 - A ColorInterpolator uses a Material target
 - A TransparencyInterpolator uses a TransparencyAttributes target
 - A SwitchValueInterpolator uses a Switch target

```
private static void createContent(TransformGroup scene_TG) {
    // create baseTG and add shape to it
    TransformGroup baseTG = new TransformGroup();
    baseTG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);
    baseTG.addChild(ShapeIndexedQuad.indexedQuadShape(0.6f));

    Alpha alpha = new Alpha(-1, Alpha.INCREASING_ENABLE |
        Alpha.DECREASING_ENABLE, 0, 0, 0, 2000, 1000, 4000, 2000, 1000);
    Transform3D axisPosition = new Transform3D();
    axisPosition.rotZ(Math.PI / 2.0);
    PositionInterpolator positionInterpol = new PositionInterpolator(alpha,
        baseTG, axisPosition, -0.6f, 0.6f); // create interpolator
    positionInterpol.setSchedulingBounds(CommonsXY.boundT);

    scene_TG.addChild(baseTG);
    scene_TG.addChild(positionInterpol);
}
```



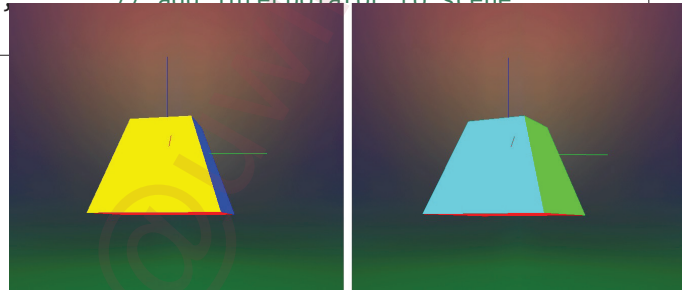
```

private static void createContent(TransformGroup scene_TG) {
    // create baseTG and add shape to it
    TransformGroup baseTG = new TransformGroup();
    baseTG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);
    baseTG.addChild(ShapeIndexedQuad.indexedQuadShape(0.6f));

    Alpha alpha = new Alpha(-1, Alpha.INCREASING_ENABLE |
        Alpha.DECREASING_ENABLE, 0, 0, 0, 2000, 1000, 4000, 2000, 1000);
    RotationInterpolator rotateInterpol = new RotationInterpolator(alpha, baseTG);
    // create interpolator
    rotateInterpol.setSchedulingBounds(CommonsXY.boundT);

    scene_TG.addChild(baseTG); // add baseTG to scene
    scene_TG.addChild(rotateInterpol); // add interpolator to scene
}

```



```

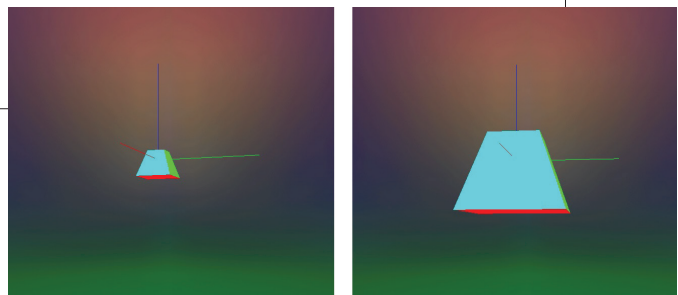
private static void createContent(TransformGroup scene_TG) {
    // create baseTG and add shape to it
    TransformGroup baseTG = new TransformGroup();
    baseTG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);
    baseTG.addChild(ShapeIndexedQuad.indexedQuadShape(0.6f));

    Alpha alpha = new Alpha(-1, Alpha.INCREASING_ENABLE |
        Alpha.DECREASING_ENABLE, 0, 0, 0, 2000, 1000, 4000, 2000, 1000);
    Transform3D axisPosition = new Transform3D();
    axisPosition.rotZ(Math.PI / 2.0);
    ScaleInterpolator scaleInterpol = new ScaleInterpolator(alpha, baseTG,
        axisPosition, 0.0f, 1.0f);

    scaleInterpol.setSchedulingBounds(CommonsXY.boundT);

    scene_TG.addChild(baseTG);
    scene_TG.addChild(scaleInterpol);
}

```



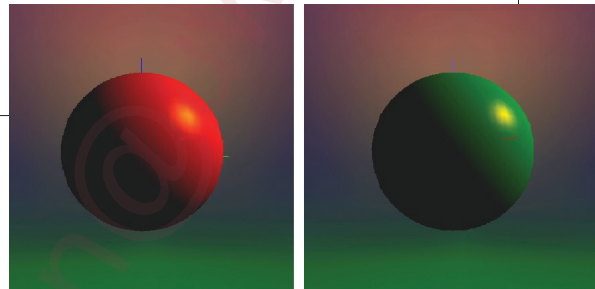

```

private static void createContent(TransformGroup scene_TG) {
    // create baseTG and add shape to it
    TransformGroup baseTG = new TransformGroup();
    Material mat = AppearanceExtra.setMaterial();
    mat.setCapability(Material.ALLOW_COMPONENT_WRITE);
    Sphere coloredSphere = new Sphere(0.4f, Sphere.GENERATE_NORMALS, 80);
    Appearance app = new Appearance();
    app.setMaterial(mat);
    coloredSphere.setAppearance(app);
    baseTG.addChild(coloredSphere);

    Alpha alpha = new Alpha(-1, Alpha.INCREASING_ENABLE |
        Alpha.DECREASING_ENABLE, 0, 0, 0, 2000, 1000, 4000, 2000, 1000);
    ColorInterpolator colorInterpol = new ColorInterpolator(alpha, mat);
    colorInterpol.setStartColor(CommonsXY.Red);
    colorInterpol.setEndColor(CommonsXY.Cyan);
    colorInterpol.setSchedulingBounds(CommonsXY.boundT);

    scene_TG.addChild(baseTG);
    scene_TG.addChild(colorInterpol);
}

```



```

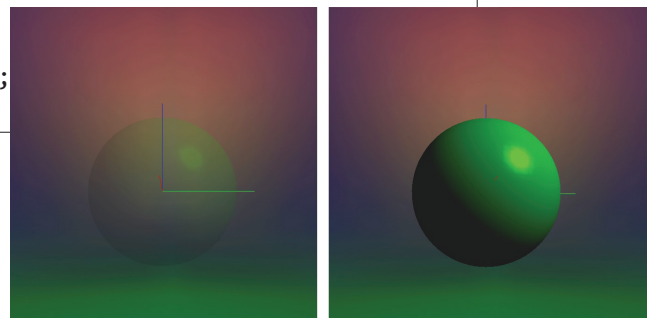
private static void createContent(TransformGroup scene_TG) {
    // create baseTG and add shape to it
    TransformGroup baseTG = new TransformGroup();
    Sphere transparentSphere = AppearanceExtra.materializeSphere(0.4f, 80);
    baseTG.addChild(transparentSphere);

    Appearance transparentApp = transparentSphere.getAppearance();
    TransparencyAttributes transparencyAtt = new TransparencyAttributes();
    transparencyAtt.setCapability(TransparencyAttributes.ALLOW_VALUE_WRITE);
    transparencyAtt.setTransparencyMode(TransparencyAttributes.FASTEST);
    transparentApp.setTransparencyAttributes(transparencyAtt);

    Alpha alpha = new Alpha(-1, Alpha.INCREASING_ENABLE |
        Alpha.DECREASING_ENABLE, 0, 0, 0, 2000, 1000, 4000, 2000, 1000);
    TransparencyInterpolator transparencyInterpol =
        new TransparencyInterpolator(alpha, transparencyAtt, 0f, 1f);
    transparencyInterpol.setSchedulingBounds(CommonsXY.boundT);

    scene_TG.addChild(baseTG);
    scene_TG.addChild(transparencyInterpol);
}

```



```

private static void createContent(TransformGroup scene_TG) {
    // create baseTG and add shape to it
    TransformGroup baseTG = new TransformGroup();

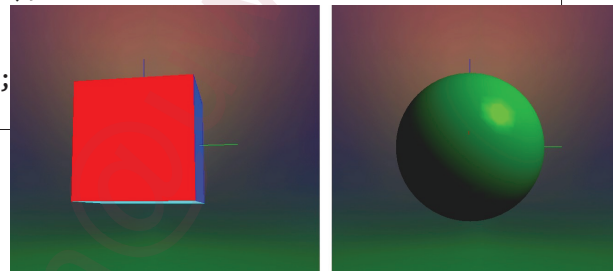
    Alpha alpha = new Alpha(-1, Alpha.INCREASING_ENABLE |
        Alpha.DECREASING_ENABLE, 0, 0, 0, 2000, 1000, 4000, 2000, 1000);
    Switch objSwitch = new Switch();
    objSwitch.setCapability(Switch.ALLOW_SWITCH_WRITE);
    SwitchValueInterpolator switchInterpol =
        new SwitchValueInterpolator(alpha, objSwitch);
    switchInterpol.setSchedulingBounds(CommonsXY.boundT);

    objSwitch.addChild(new ColorCube(0.3f));
    objSwitch.addChild(AppearanceExtra.materializeSphere(0.4f, 60));

    baseTG.addChild(objSwitch);
    switchInterpol.setLastChildIndex(1);

    scene_TG.addChild(baseTG);
    scene_TG.addChild(switchInterpol);
}

```



```

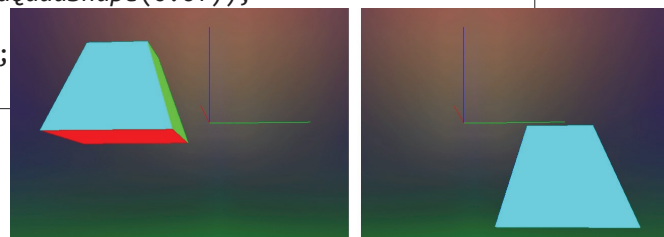
private static void createContent(TransformGroup scene_TG) {
    Point3f[] points = new Point3f[19]; points[18] = new Point3f(0f, 0.5f, 0f);
    for (int i = 0; i < 18; i++) {
        float x = (float) 0.5 * (float) Math.sin((float) Math.PI * 2 * i / 17);
        float y = (float) 0.5 * (float) Math.cos((float) Math.PI * 2 * i / 17);
        points[i] = new Point3f(x, y, 0); } // define position path
    float[] knots = new float[19]; knots[18] = 1.0f;
    for (int i = 0; i < 18; i++) knots[i] = 1.0f * i / 17f;

    TransformGroup baseTG = new TransformGroup(); // allow 'baseTG' to change
    baseTG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);

    Alpha alpha = new Alpha(-1, Alpha.INCREASING_ENABLE |
        Alpha.DECREASING_ENABLE, 0, 0, 0, 2000, 1000, 4000, 2000, 1000);
    Transform3D axis_t = new Transform3D();
    PositionPathInterpolator positionPathInterpol =
        new PositionPathInterpolator(alpha, baseTG, axis_t, knots, points);
    positionPathInterpol.setSchedulingBounds(CommonsXY.boundT);

    baseTG.addChild(ShapeIndexedQuad.indexedQuadShape(0.6f));
    scene_TG.addChild(baseTG);
    scene_TG.addChild(positionPathInterpol);
}

```



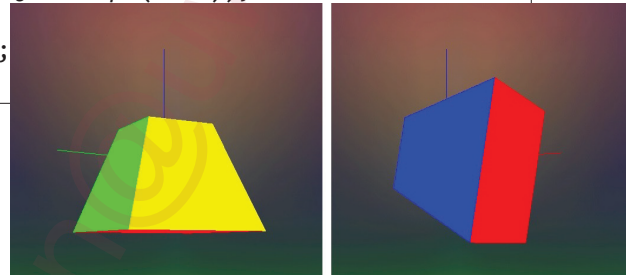
```

private static void createContent(TransformGroup scene_TG) {
    float[] knots = { 0.0f, 0.25f, 0.5f, 0.75f, 1.0f };
    Quat4f[] quats = {new Quat4f(0.0f, -1.0f, 0.0f, 1.0f),
        new Quat4f(0.0f, 1.0f, 0.0f, 1.0f), // define path with quaternion
        new Quat4f(1.0f, 0.0f, 0.0f, 1.0f),
        new Quat4f(-1.0f, 0.0f, 0.0f, 1.0f),
        new Quat4f(0.0f, -1.0f, 0.0f, 1.0f)};

    TransformGroup baseTG = new TransformGroup(); // allow 'baseTG' to change
    baseTG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);
    Alpha alpha = new Alpha(-1, Alpha.INCREASING_ENABLE |
        Alpha.DECREASING_ENABLE, 0, 0, 0, 2000, 1000, 4000, 2000, 1000);
    Transform3D coordRot = new Transform3D(); // default local coordinate system
    RotationPathInterpolator rotationPathInterpol =
        new RotationPathInterpolator(alpha, baseTG, coordRot, knots, quats);
    rotationPathInterpol.setSchedulingBounds(CommonsXY.boundT);

    baseTG.addChild(ShapeIndexedQuad.indexedQuadShape(0.6f));
    scene_TG.addChild(baseTG);
    scene_TG.addChild(rotationPathInterpol);
}

```



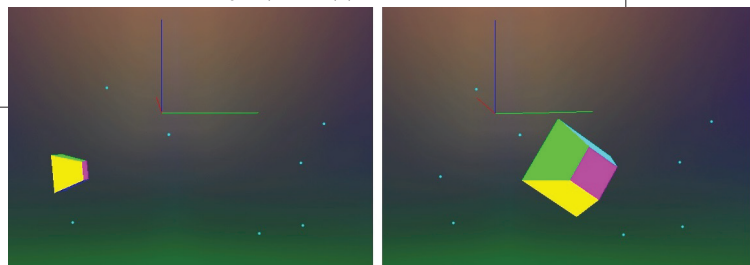
```

private static void createContent(TransformGroup scene_TG) {
    float[] knots = { 0.0f, 0.1f, 0.2f, 0.3f, 0.4f, 0.6f, 0.8f, 0.9f, 1.0f };
    Quat4f[] quats = new Quat4f[9]; Point3f[] positions = new Point3f[9];
    scene_TG.addChild(visiblePoints(quats, positions)); // visible points

    TransformGroup baseTG = new TransformGroup(); // allow 'baseTG' to change
    baseTG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);
    Transform3D axisOfRotPos = new Transform3D();
    AxisAngle4f axis = new AxisAngle4f(1.0f, 0.0f, 0.0f, 0.0f);
    axisOfRotPos.set(axis);
    Alpha alpha = new Alpha(-1, Alpha.INCREASING_ENABLE |
        Alpha.DECREASING_ENABLE, 0, 0, 0, 2000, 1000, 4000, 2000, 1000);
    RotPosPathInterpolator rotPosPath = new RotPosPathInterpolator(alpha,
        baseTG, axisOfRotPos, knots, quats, positions);
    rotPosPath.setSchedulingBounds(CommonsXY.boundT);

    baseTG.addChild(ShapeIndexedQuad.indexedQuadShape(0.6f));
    scene_TG.addChild(baseTG);
    scene_TG.addChild(rotPosPath);
}

```



```

public static Shape3D visiblePoints(Quat4f[] quats, Point3f[] positions) {
    quats[8] = quats[0] = new Quat4f(0.0f, 1.0f, 1.0f, 0.0f);
    quats[1] = new Quat4f(1.0f, 0.0f, 0.0f, 0.0f);
    quats[2] = new Quat4f(0.0f, 1.0f, 0.0f, 0.0f);
    quats[3] = new Quat4f(0.0f, 1.0f, 1.0f, 0.0f);
    quats[4] = new Quat4f(0.0f, 0.0f, 1.0f, 0.0f);
    quats[5] = new Quat4f(0.0f, 1.0f, 1.0f, 0.0f);
    quats[6] = new Quat4f(1.0f, 1.0f, 0.0f, 0.0f);
    quats[7] = new Quat4f(1.0f, 0.0f, 0.0f, 0.0f);
    positions[8] = positions[0] = new Point3f(0.0f, 0.0f, -1.0f);
    positions[1] = new Point3f(0.7f, -0.7f, -2.0f);
    positions[2] = new Point3f(-0.7f, 0.7f, -3.0f);
    positions[3] = new Point3f(1.4f, 0.0f, -4.0f);
    positions[4] = new Point3f(-1.4f, -0.7f, -5.0f);
    positions[5] = new Point3f(2.1f, 0.7f, -6.0f);
    positions[6] = new Point3f(-2.1f, 0.0f, -7.0f);
    positions[7] = new Point3f(1.4f, -0.7f, -4.0f);

    PointArray point_geom = new PointArray(9, GeometryArray.COORDINATES);
    point_geom.setCoordinates(0, positions);
    Appearance points_appear = new Appearance();
    ColoringAttributes points_coloring = new ColoringAttributes();
    points_coloring.setColor(CommonsXY.Lime);
    points_appear.setColoringAttributes(points_coloring);
    PointAttributes points_points = new PointAttributes(4.0f, true);
    points_appear.setPointAttributes(points_points);

    return new Shape3D(point_geom, points_appear);
}

```

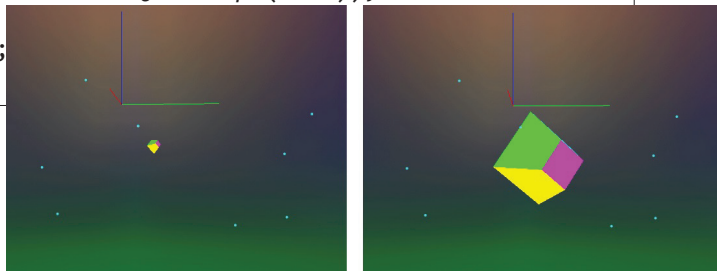
```

private static void createContent(TransformGroup scene_TG) {
    float[] knots = { 0.0f, 0.1f, 0.2f, 0.3f, 0.4f, 0.6f, 0.8f, 0.9f, 1.0f };
    float[] scales = { 1.0f, 0.5f, 0.1f, 0.5f, 1.0f, 0.5f, 0.1f, 0.5f, 1.0f };
    Quat4f[] quats = new Quat4f[9]; Point3f[] positions = new Point3f[9];
    scene_TG.addChild(BehaviorRotPosPath.visiblePoints(quats, positions));

    TransformGroup baseTG = new TransformGroup(); // allow 'baseTG' to change
    baseTG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);
    Transform3D axisOfRotPos = new Transform3D();
    AxisAngle4f axis = new AxisAngle4f(1.0f, 0.0f, 0.0f, 0.0f);
    axisOfRotPos.set(axis);
    Alpha alpha = new Alpha(-1, Alpha.INCREASING_ENABLE |
        Alpha.DECREASING_ENABLE, 0, 0, 0, 2000, 1000, 4000, 2000, 1000);
    RotPosScalePathInterpolator rotPosScale = new RotPosScalePathInterpolator(
        alpha, baseTG, axisOfRotPos, knots, quats, positions, scales);
    rotPosScale.setSchedulingBounds(CommonsXY.boundT);

    baseTG.addChild(ShapeIndexedQuad.indexedQuadShape(0.6f));
    scene_TG.addChild(baseTG);
    scene_TG.addChild(rotPosScale);
}

```



PART III. SPECIALIZED BEHAVIORS

1. Some specialized behaviors are provided by Java 3D

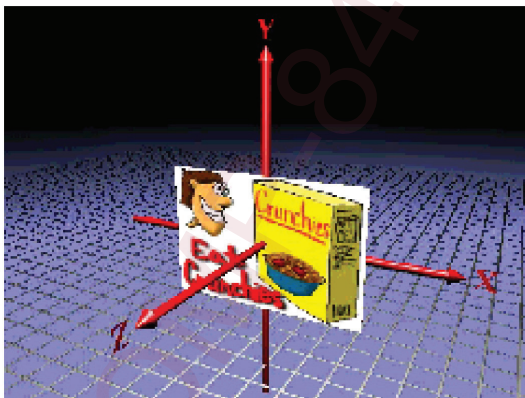
- *Billboard* auto-rotation of shapes to face the viewer
- Switching between shapes based upon viewer's distance
- Specialized behaviors are extensions of Behavior

Class Hierarchy

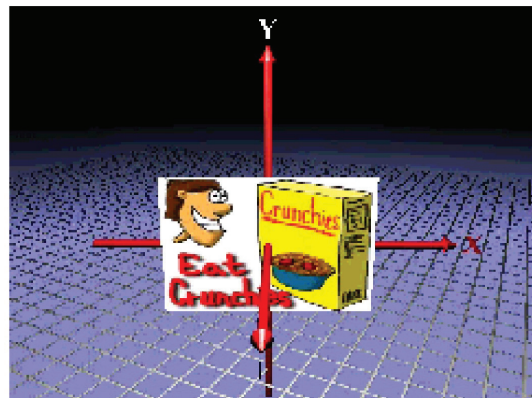
```
java.lang.Object
├─ org.jogamp.java3d.SceneGraphObject
│   └─ org.jogamp.java3d.Node
│       └─ org.jogamp.java3d.Leaf
│           └─ org.jogamp.java3d.Behavior
│               ├── org.jogamp.java3d.Billboard
│               ├── org.jogamp.java3d.LOD
│               └─ org.jogamp.java3d.DistanceLOD
```

2. A Billboard is a specialized behavior that:

- Tracks the ViewPlatform
- Generates a rotation about an axis so that the Z-axis always points at the platform



Viewer steps to the right . . .

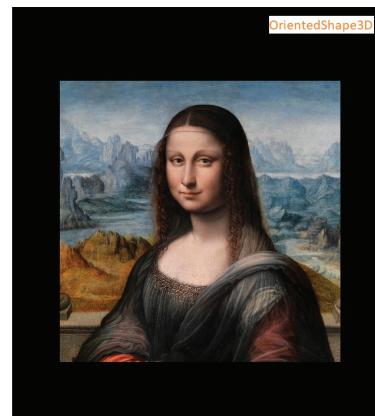
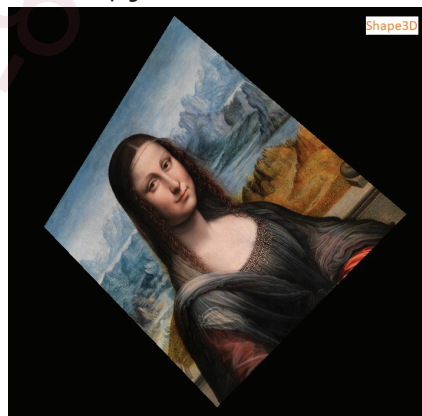


. . . and the behavior immediately rotates the shape

3. `OrientedShape3D` objects are used to perform the same function as the `Billboard` behavior.

- `OrientedShape3D` differs from `Billboard`:
 - It works for applications with more than one view.
 - `OrientedShape3D` objects can be a shared object.
 - It is a subclass of `Shape3D`, not a behavior, and hence does not have a scheduling bounds to consider.
- It can be used for all `Billboard` applications.
 - `Billboard` is not deprecated for the reason of backward compatibility with existing applications.
- An `OrientedShape3D` can rotate about an axis or a point.
 - The `OrientedShape3D` object orients in a way to face its positive z-axis to the viewer.

- If an axis is specified in parallel to the Z axis, the `OrientedShape3D` will simply not rotate.
- If the alignment mode is `ROTATE_ABOUT_POINT`, the rotation will be about the specified point, with an additional rotation to align the +y axis of the `TransformGroup` with the +y axis in the View.



```

private static OrientedShape3D oriented_plane(float w, float h, float s) {
    Point3f[] verts = { new Point3f(-w * s, -h * s, 0f),
        new Point3f(w * s, -h * s, 0f), new Point3f(w * s, h * s, 0f),
        new Point3f(w * -s, h * s, 0f) };

    QuadArray qa = new QuadArray(4,
        GeometryArray.COORDINATES |
        GeometryArray.COLOR_3 |
        GeometryArray.TEXTURE_COORDINATE_2);
    qa.setCoordinates(0, verts);
    float uv0[] = { 0f, 0f };
    float uv1[] = { 1f, 0f };
    float uv2[] = { 1f, 1f };
    float uv3[] = { 0f, 1f };
    qa.setTextureCoordinate(0, 0, uv0);
    qa.setTextureCoordinate(0, 1, uv1);
    qa.setTextureCoordinate(0, 2, uv2);
    qa.setTextureCoordinate(0, 3, uv3);

    OrientedShape3D oriented_plane = new OrientedShape3D(qa,
        AppearanceTextureAttr.texturedApp(),
        OrientedShape3D.ROTATE_ABOUT_POINT, new Point3f(0f, 0f, 0f));
    return oriented_plane;
}

```

4. *Level of Detail* (LOD) varies the level of detail

- LOD is a specialized behavior that:
 - Tracks the ViewPlatform
 - Computes a distance to a shape
 - Maps the distance to Switch group child choices
- LOD is a technique to improve performance
 - It reduces the complexity of a visual object without affecting the visual result.
 - It helps to save a significant computational time without visual loss of content.
- The LOD abstract class generalizes level-of-detail behaviors, and the DistanceLOD class implements distance-based switching level-of-detail.


```

private static BranchGroup createContent() {
    BranchGroup baseBG = new BranchGroup(); addTwoCylinders(baseBG);

    Alpha alpha = new Alpha(-1, Alpha.INCREASING_ENABLE | Alpha.DECREASING_ENABLE,
        0, 0, 5000, 1000, 1000, 5000, 1000, 1000);
    TransformGroup translateTG = new TransformGroup();
    translateTG.setCapability(TransformGroup.ALLOW_TRANSFORM_WRITE);
    AxisAngle4f axisTrans = new AxisAngle4f(0.0f, 1.0f, 0.0f, (float) Math.PI / -2.0f);
    Transform3D T3D = new Transform3D(); T3D.set(axisTrans);
    PositionInterpolator positionInt = // define behavior for translation along axis
        new PositionInterpolator(alpha, translateTG, T3D, -2.5f, -15.0f);
    positionInt.setSchedulingBounds(CommonsXY.boundT);

    float[] distances = { 5.0f, 7.5f, 12.0f };
    Switch SwitchTarget = new Switch(); // create DistanceLOD target object
    SwitchTarget.setCapability(Switch.ALLOW_SWITCH_WRITE);
    SwitchTarget.addChild(new Cylinder(0.9f, 0.9f, setApp(CommonsXY.Green)));
    SwitchTarget.addChild(new Cylinder(0.75f, 0.75f, setApp(CommonsXY.Blue)));
    SwitchTarget.addChild(new Cylinder(0.5f, 0.5f, setApp(CommonsXY.Orange)));
    SwitchTarget.addChild(new Cylinder(0.25f, 0.25f, setApp(CommonsXY.Red)));
    DistanceLOD distanceLOD = new DistanceLOD(distances, new Point3f());
    distanceLOD.addSwitch(SwitchTarget);
    distanceLOD.setSchedulingBounds(CommonsXY.boundT);
    if ((SwitchTarget.numChildren() - 1) != distanceLOD.numDistances()) {
        System.out.println("Error in creating LOD ");
    }

    baseBG.addChild(translateTG); baseBG.addChild(positionInt);
    translateTG.addChild(distanceLOD); translateTG.addChild(SwitchTarget);

    return baseBG; }

```

```

private static void addTwoCylinders(BranchGroup baseBG) {
    Transform3D sCylinder = new Transform3D(); // object at a nearer distance
    sCylinder.set(new Vector3f(0.7f, 0.0f, 0.0f));
    TransformGroup smallCylinder = new TransformGroup(sCylinder);
    baseBG.addChild(smallCylinder);
    smallCylinder.addChild(new Cylinder(0.25f, 0.25f, setApp(CommonsXY.Green)));

    sCylinder.set(new Vector3f(-6.0f, 0.0f, -12.0f)); // the object at further
    TransformGroup smallCylinder1 = new TransformGroup(sCylinder);
    baseBG.addChild(smallCylinder1);
    smallCylinder1.addChild(new Cylinder(0.9f, 0.9f, setApp(CommonsXY.Red)));
}

private static Appearance setApp(Color3f clr) {
    Appearance app = new Appearance();
    ColoringAttributes colorAtt = new ColoringAttributes();
    colorAtt.setColor(clr);
    app.setColoringAttributes(colorAtt);
    return app;
}

```

